

QA & SOFTWARE TESTING

Comprehensive Study Material

Last Updated: June 30, 2026

TABLE OF CONTENTS

1. Testing Fundamentals
2. Test Levels
3. Test Types
4. Test Case Design
5. Defect Management
6. Test Automation
7. Performance Testing
8. Security Testing
9. Test Management
10. Tools and Frameworks

1. TESTING FUNDAMENTALS

Quality Assurance vs Quality Control: QA focuses on process improvement to prevent defects. QC focuses on identifying defects in products. QA is proactive, QC is reactive.

Testing Objectives: Find bugs before production. Verify requirements met. Ensure reliability and performance. Build confidence in product quality.

Test Lifecycle: Planning, design, execution, reporting, closure. Iterative process with feedback. Each phase has specific deliverables and activities.

Testing Principles: Testing shows presence of defects, not absence. Impossible to test everything. Early testing saves time and cost. Test for both valid and invalid inputs.

2. TEST LEVELS

Unit Testing: Tests individual components or functions. Developers write tests. Fast and cheap. Catches logical errors early. White-box testing approach.

Integration Testing: Tests component interactions and interfaces. Verifies data flow between modules. Big-bang, top-down, bottom-up strategies. Gray-box testing.

System Testing: Tests complete application against requirements. End-to-end testing. Functional and non-functional aspects. Black-box testing approach.

User Acceptance Testing: Users test if system meets business requirements. Final validation before release. Identifies business logic issues. Alpha and beta testing.

3. TEST TYPES

Functional Testing: Verifies features work correctly. Tests inputs and outputs. Validates against specifications. Most common testing type.

Non-Functional Testing: Tests performance, security, usability, reliability. Addresses how system behaves. Load testing, stress testing, security testing.

Regression Testing: Re-tests existing features after changes. Ensures fixes don't break functionality. Automation critical due to repetitive nature.

Smoke Testing: Quick test of critical functionality. Identifies basic defects early. Gating test before detailed testing. Saves time if major issues found.

Sanity Testing: Validates specific functionality after code changes. Narrows focus on changed areas. Less comprehensive than smoke testing.

4. TEST CASE DESIGN

Test Case Structure: Consists of inputs, preconditions, steps, expected results. Unique ID and description. Written before or during development. Repeatable and clear.

Boundary Value Analysis: Tests at boundaries of input ranges. Tests minimum, maximum, just inside, just outside. Catches off-by-one errors. Effective for ranges.

Equivalence Partitioning: Divides inputs into classes where all behave similarly. Tests representative from each class. Reduces test cases while maintaining coverage.

Decision Table Testing: Documents inputs, conditions, and expected outputs. Systematic coverage of combinations. Useful for business logic with multiple conditions.

State Transition Testing: Tests valid and invalid state transitions. Documents states and transitions. Ensures system behaves correctly in each state.

5. DEFECT MANAGEMENT

Defect Lifecycle: New (reported), assigned, in progress (fixing), resolved, verified (fixed), closed. Tracking enables accountability. Reopened if verification fails.

Defect Severity: Critical (system down), major (significant impact), minor (workaround possible), trivial (cosmetic). Priority based on business value.

Defect Report: Includes description, steps to reproduce, expected vs actual, screenshots, environment details. Completeness aids resolution.

Metrics: Defect density (defects per KLOC), escape rate (production defects), removal efficiency. Track trends. Identify quality issues.

6. TEST AUTOMATION

Automation Benefits: Repeatable, faster execution, 24/7 testing, regression testing efficiency. Reduces manual effort. Enables continuous integration.

When to Automate: Repetitive tests, regression tests, performance tests. Not suitable for exploratory or one-time tests. ROI analysis justifies investment.

Test Automation Framework: Defines structure, utilities, standards. Modular approach for maintainability. Keyword-driven, data-driven, or hybrid frameworks.

Challenges: High initial cost, maintenance overhead, tool selection, false positives. Requires discipline and skilled automation engineers.

7. PERFORMANCE TESTING

Load Testing: Simulates expected user load. Measures response time and system behavior. Identifies maximum capacity. Ensures acceptable performance.

Stress Testing: Pushes system beyond normal capacity. Identifies breaking point. Tests recovery. Validates error handling under extreme conditions.

Spike Testing: Sudden traffic increase. Tests system resilience. Measures time to handle spike. Simulates real-world traffic patterns.

Endurance Testing: Sustained load over extended period. Identifies memory leaks and resource exhaustion. Verifies stability. Mimics production usage.

Metrics: Response time, throughput, resource utilization. Baseline for comparison. Track trends. Identify bottlenecks and optimization opportunities.

8. SECURITY TESTING

Security Testing Objectives: Identify vulnerabilities and security gaps. Verify authentication/authorization. Ensure data protection. Validate input validation.

SQL Injection: Malicious SQL through user input. Tests input validation. Can extract or modify data. Critical vulnerability.

Cross-Site Scripting (XSS): Injects malicious scripts into web pages. Tests output encoding. Can steal session cookies. Common web vulnerability.

Authentication Testing: Verifies user identity correctly. Tests password policies, session management. Validates credentials handling.

Penetration Testing: Simulates attacker perspective. Attempts to break system. Identifies exploitable vulnerabilities. Expensive but thorough.

9. TEST MANAGEMENT

Test Planning: Defines scope, resources, schedule, risks. Estimates effort. Identifies testing strategy. Foundation for successful testing.

Test Coverage: Percentage of code/requirements tested. 100% impossible and impractical. Aim for meaningful coverage (70-80%). Tools measure coverage.

Test Reporting: Progress reports, status dashboards, metrics. Executive summaries with trends. Enables data-driven decisions.

Risk-Based Testing: Prioritizes testing based on risk. High-risk areas get more testing. Optimizes resource allocation. Maximizes quality within constraints.

10. TOOLS AND FRAMEWORKS

Selenium: Open-source browser automation. Supports multiple languages. Wide adoption. Cloud integration. Most popular web automation tool.

JUnit/TestNG: Java testing frameworks. Annotation-based test organization. Assertion libraries for validations. Continuous integration friendly.

Pytest: Python testing framework. Simple syntax. Plugins for fixtures and markers. Lightweight and powerful.

LoadRunner/JMeter: Performance testing tools. Simulate virtual users. Measure performance metrics. JMeter free and open-source.

JIRA: Defect tracking and test management. Integrates with development tools. Workflow customization. Industry standard.